

System: EZ-Link machine (those found at MRTs)

Use case: UC2 top-up EZ-Link card

Actor: EZ-Link card user

Preconditions: All hardware in working order.

MSS:

1. User places the card on the reader.
2. System displays card details and prompts for desired action.
3. User selects top-up.
4. System requests for top-up details (amount, payment option, etc.).
5. User enters details.
6. System processes cash payment (UC02) or NETS payment (UC03).
7. System updates the card value and indicates transaction as completed.
8. User removes the card.

Use case ends.

Extensions:

*a. User removed card.

*a1. System indicates the transaction has been aborted.

Use case ends.

- a. It does not consider 'system crashes'
- b. It does not contain enough UI details.
- c. The extension given is in fact an inclusion.
- d. No post-conditions are given.
- e. 'Use case ends' is duplicated.

How many of the above are
problems in the given use case?

0 1 2 3 4 5

➤ Revisit last week's
topics using
examples/questions

☐ Preview of next week's
topics

Q₁

Canvas quiz for Week 8: Part 1

System: EZ-Link machine (those found at MRTs)

Use case: UC2 top-up EZ-Link card

Actor: EZ-Link card user

Preconditions: All hardware in working order.

MSS:

1. User places the card on the reader.
2. System displays card details and prompts for desired action.
3. User selects top-up.
4. System requests for top-up details (amount, payment option, etc.).
5. User enters details.
6. System processes cash payment (UC02) or NETS payment (UC03).
7. System updates the card value and indicates transaction as completed.
8. User removes the card.

Use case ends.

Extensions:

*a. User removed card.

*a1. System indicates the transaction has been aborted.

Use case ends.

- | |
|---|
| a. It does not consider 'system crashes' |
| b. It does not contain enough UI details. |
| c. The extension given is in fact an inclusion. |
| d. No post-conditions are given. |
| e. 'Use case ends' is duplicated. |

How many of the above are
problems in the given use case?

0 1 2 3 4 5

Extensions: alternative diversions from the current path.

Inclusions: part of the current path, but details are elsewhere.

What if we ...?

made a mistake in use case step numbering?

misclassified a non-functional requirement?

had an unnecessary term in the Glossary?

didn't phrase a user story correctly?

put a UI detail in the use case?

missed an important user story?

missed an important NFR?

**Some are bigger
problems than others!**

Requirements are rather subjective! ☹️

In the exam: We'll try to set the question to be less subjective.

In real projects ...

Make a **judgement call**,
based on potential cost/risk.

Also, **cross-verify** with stakeholders.

Q₂

Question about iP: When should you save data to the disk?

- a. Only if the user gives a command to save data
- b. When the application is closed
- c. After every user command
- d. After every user command that changes data

Fewer writes.

Higher risk of data loss.

More writes.

No risk of data loss.

Worried about UI lag?

Use threads to prevent UI lag?

Beware of
premature
optimization!

Worried about saving data even after an incorrect operation by the user? Can mitigate by providing an 'undo' command

[This will not affect your iP grading though]

Changing the signature of a public method of a class can affect...

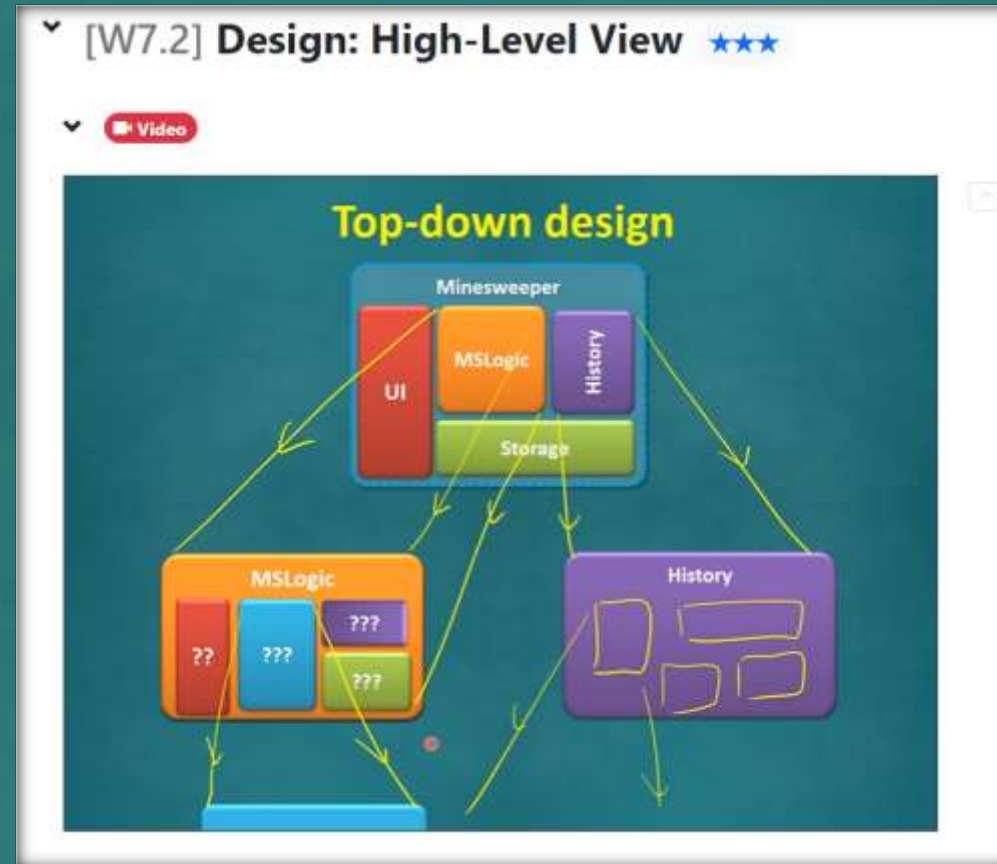
- | |
|---------------------------------------|
| ☐ abstraction |
| ☐ coupling |
| ☐ cohesion |
| ☐ dependencies |

```
class TaskList  
init(List<Task> tasks) → init(File tasksFile)
```

Whoa! Are we going to get this kind of questions in the exam?
No 'tricky' questions in the exam.

Question about iP: *Which design approach was used?*

- top-down
 - bottom-up
 - sandwich
 - agile
- (Opposite: full design upfront)



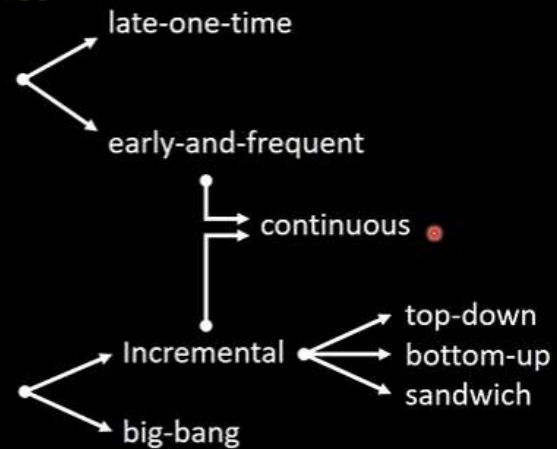
tP is using which *integration* approach?

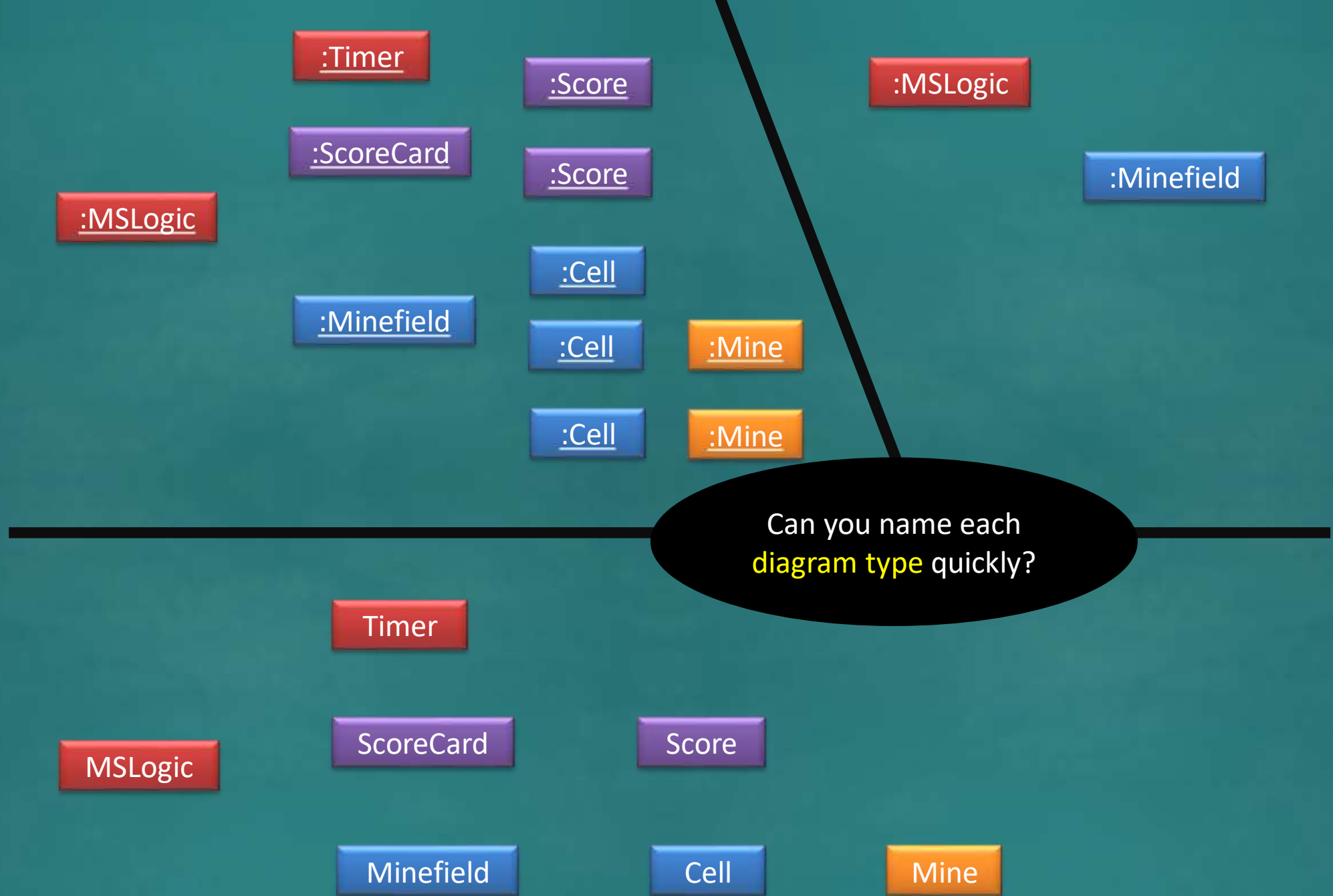
- top-down
- bottom-up
- sandwich
- big bang
- continuous

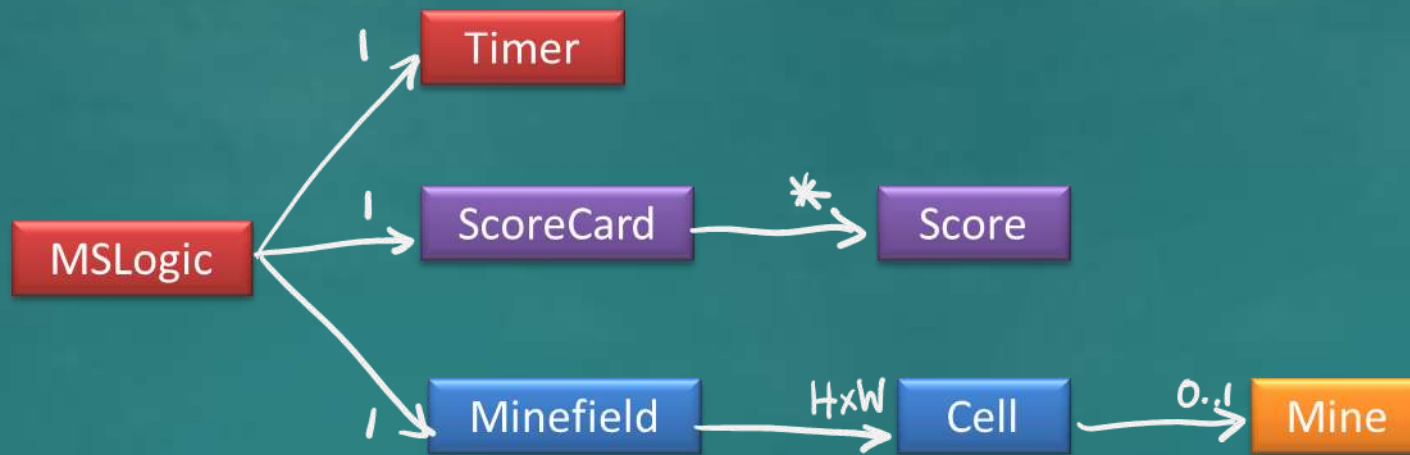
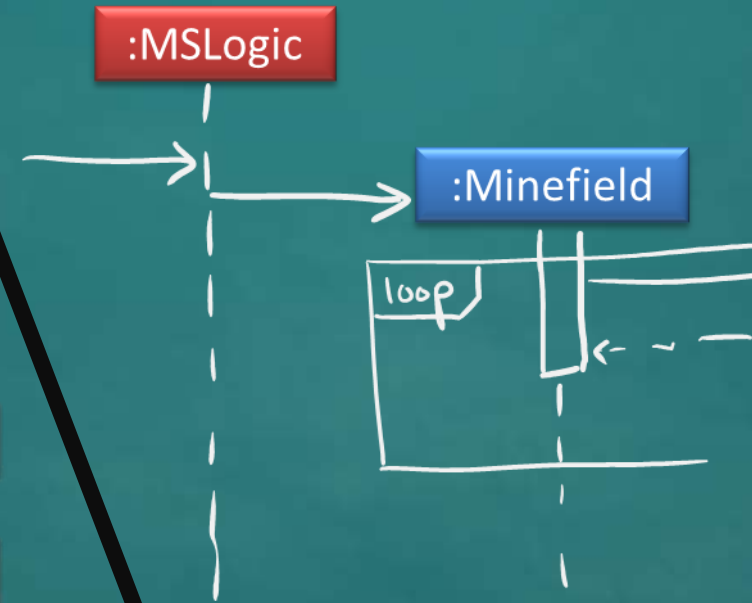
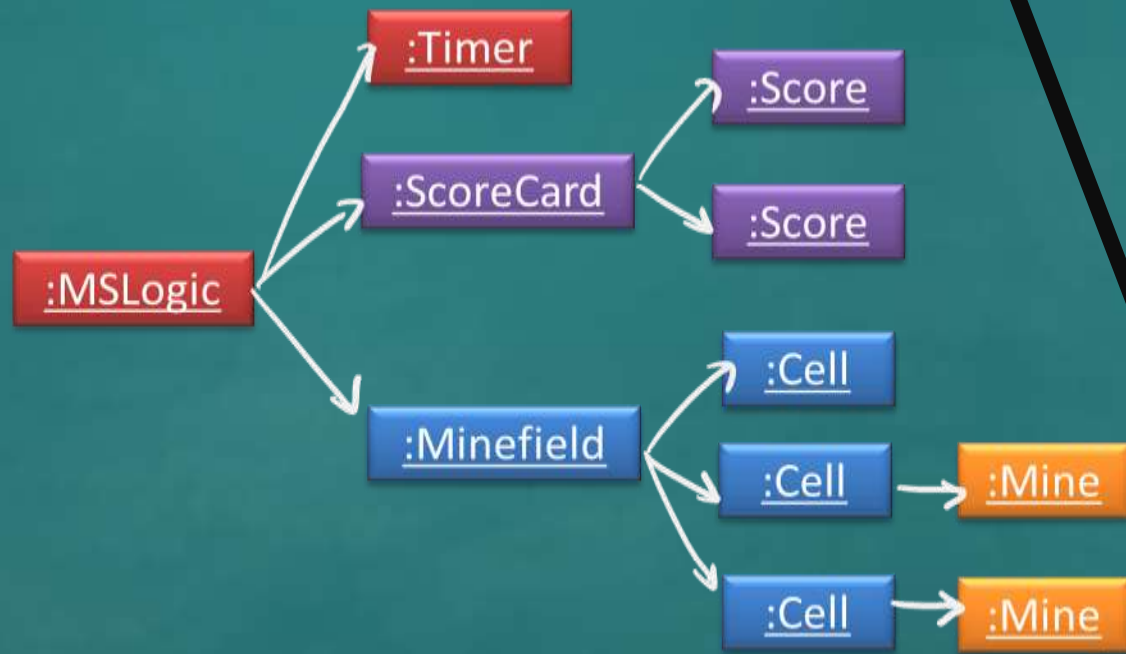
▼ [W7.5] Integration Approaches ★★

▼ Video

- Implementation
→ Integration
→ Approaches

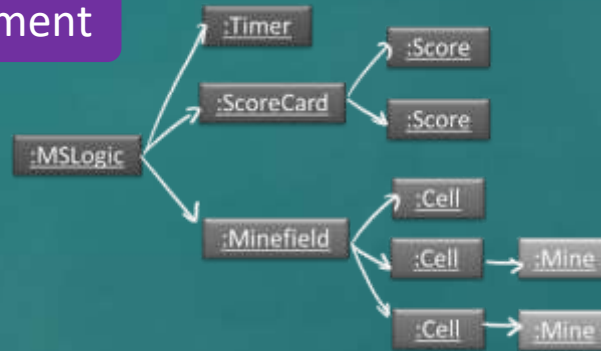






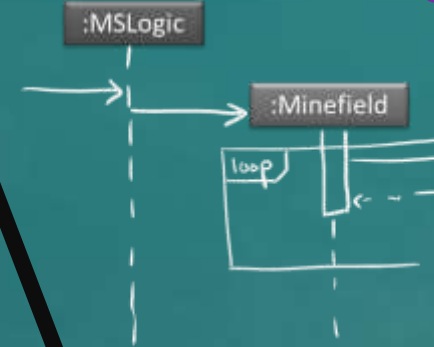
Object diagram

A moment



Sequence diagram

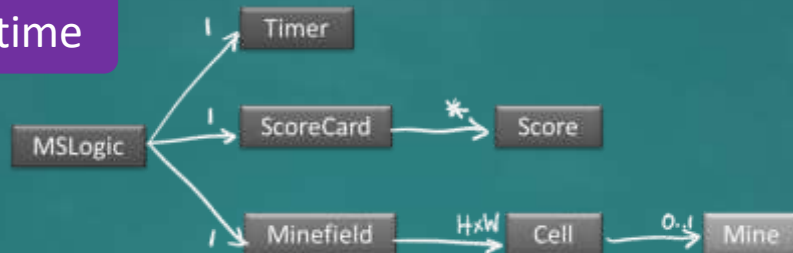
period



Can you remember
what **each diagram type**
represents?

Class diagram

All the time



Object diagram

A moment



Like a photo taken of a game which shows the current state of the game e.g., shows who is in the field, who has the ball, current scoreboard

Sequence diagram

period

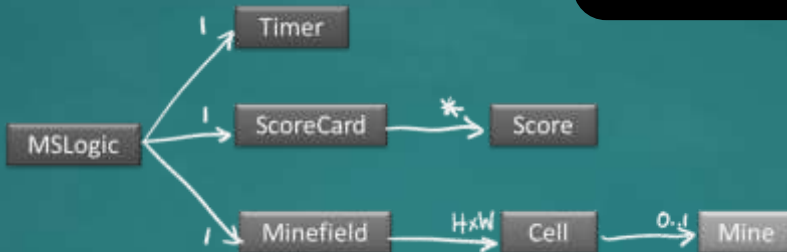


Like a strategy for scoring a goal e.g., who takes the ball first, and pass to whom next, etc.

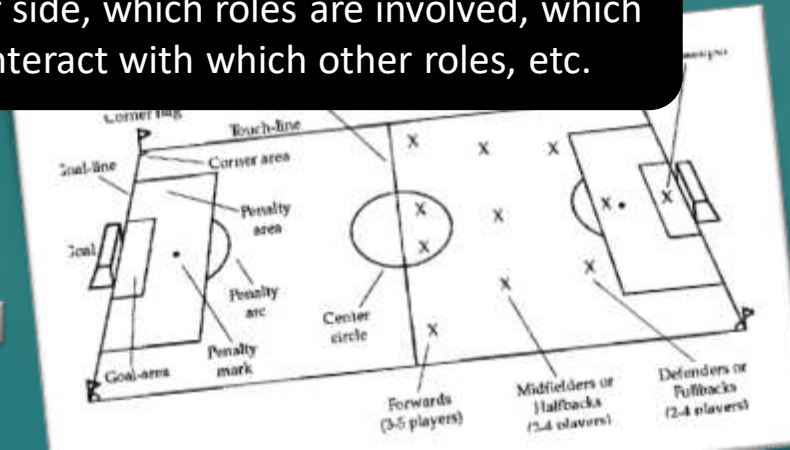
Football analogy

Class diagram

All the time



Like the rules of the game e.g., how many players per side, which roles are involved, which roles can interact with which other roles, etc.



✓ Revisit last week's topics
using examples/questions

➤ Preview of
next week's topics

Drawing UML diagrams

Idea →

Code →

UML

1. Class diagrams
2. Object diagrams
3. Sequence diagrams
4. ...
5.

→ Understand ✓

Q

To use for,
[] discussion
[] documenting
[] code generation

Different **aims, needs, tools, skills**

Quick and dirty – barely enough details
draw fast, talk as you draw

Current focus

Neater (e.g., optimal layout, colors, ...),
a bit more detailed (but not a lot either).

Later in the tP

Easy to maintain: use a tool that allows incremental edits.
e.g., PlantUML

Future-proof:

- multiple smaller diagrams, not all-in-one
- omit more volatile parts
- draw at pseudo-code level
(i.e., higher level of abstraction)...

Idea →

Code →

UML

1. Class diagrams
2. Object diagrams
3. Sequence diagrams
4. ...
5.

→ Understand ✓

Week 8

Topics:

- [W8.1] [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [W8.2] [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [W8.3] [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [W8.4] [Revisiting] Drawing Types ⚡⚡

- This week's quiz has 2 parts.
- **Part 1:**
 - Covers these topics but from an interpreting/drawing perspective.
 - Can see the answer immediately after submitting.
 - Do this part before the tutorial.

Admin:

1. Submit weekly quiz [P](#)
2. Submit midterm peer evaluations on TEAMMATES [🕒 Sun, Mar 16th 2359](#) [P](#)

iP:

1. Evaluate two peer iPs [🕒 Sat, Mar 15th 2359](#) [P](#)

tP: [📦 v1.2](#)

1. 👤 Divide MVP features among members
2. 👤/👤 [Optional] Rename packages/classes
3. 👤 Add the first functionality increment

[🕒 Thu, Mar 13th 23:59](#)

Watch *some* if not all
(ideally, before the tutorial)

 Guidance for the item(s) below:

In an earlier week you learned how to interpret class diagrams and object diagrams. Let's go back to that topic but this time let's learn how to draw those diagrams to match code. You'll need to draw class/object diagrams when you document the design of your tP.

But note that while in the tP you'll be using diagramming tools to draw the diagram, here, we focus on hand drawing. The reason is, we also want you to be able to quickly hand-draw UML diagrams to aid discussions (e.g., draw a diagram on a whiteboard while explaining a design decision to colleagues).



[\[image source\]](#)

The relevant UML topics are repeated further down in this page, for reference. In addition, here are some **worked-examples/exercise of drawing class/object/sequence diagrams**:

- >  **Video**  Drawing class/object diagrams (basic) - `Box` , `Item` , `Lid`
- >  **Video** Drawing class/object diagrams (intermediate) - `Action` , `Task` , `History`
- >  **Video** Drawing class/object diagrams (intermediate) - `Person` , `Inbox` , `Message`
- >  **Video** Drawing class/object diagrams (intermediate) - `Person` , `Project` , `Task`

Week 8

1 Exercise: draw a class diagram and an object diagram

Question adapted from past exam paper.

1. **before the tutorial** Each student do the following exercise, by **hand-drawing** the answer.
Use the following layout:



2. Draw a Class Diagram for the code (`StockItem` , `Inventory` , `Review` etc.)



2. **during the tutorial**
 - Paste the diagram (take a photo if you drew on paper) in the *tutorial workspace* document.
 - **Discuss answers** as guided by the tutor.

2 Exercise: draw a sequence diagram

1. **before the tutorial** Do the following exercise similar to the previous one.

Week 8

Topics:

- [W8.1] [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [W8.2] [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [W8.3] [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [W8.4] Testing: Types ⚡⚡
- [W8.5] Testing: Intermediate Concepts ⚡⚡

Has 2 parts. Do part 1 before the tutorial

Admin:

1. Submit weekly quiz P
2. Submit midterm peer evaluations on TEAMMATES 🕒 Sun, Mar 16th 2359 P

iP:

1. Evaluate two peer iPs 🕒 Sat, Mar 15th 2359 P

tP: 📁 v1.2

1. 👤 Divide MVP features among members
2. 👤/👤 [Optional] Rename packages/classes
3. 👤 Add the first functionality increment

🕒 Thu, Mar 13th 23:59

Your tP, at the end of the semester ...





The ability to write **code that can survive the harsh realities of the real world** is a very important outcome of this module.



Practical Exam:

4-5 peers testing your app



Better to find bugs yourself!

Dry Run



Week 8

Topics:

- [W8.1] [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [W8.2] [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [W8.3] [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [W8.4] Testing: Types ⚡⚡
- [W8.5] Testing: Intermediate Concepts ⚡⚡

Admin:

1. Submit weekly quiz [P](#)
2. Submit midterm peer evaluations on TEAMMATES [🕒 Sun, Mar 16th 2359](#) [P](#)

iP:

1. Evaluate two peer iPs [🕒 Sat, Mar 15th 2359](#) [P](#)

tP: [📦 v1.2](#)

1. 👤 Divide MVP features among members
2. 👤/👤 [Optional] Rename packages/classes
3. 👤 Add the first functionality increment

[🕒 Thu, Mar 13th 23:59](#)



But it worked in
my machine!

Famous Software Project Failures – Denver Airport Project

Project goal: automate the handling of
baggage through the entire airport.

Extensive tests done, even using real
baggage belts.

But critical operations failed entirely
when they switched to the new system.

How come the bugs were not found
earlier during testing?

Reason: The testing was done using
empty bags. The sensor output was
slightly different when actual bags were
on baggage belts, which caused the
system to malfunction.

Week 8

Topics:

- [\[W8.1\]](#) [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [\[W8.2\]](#) [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [\[W8.3\]](#) [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [\[W8.4\]](#) **Testing: Types** ⚡⚡
- [\[W8.5\]](#) **Testing: Intermediate Concepts** ⚡⚡

Admin:

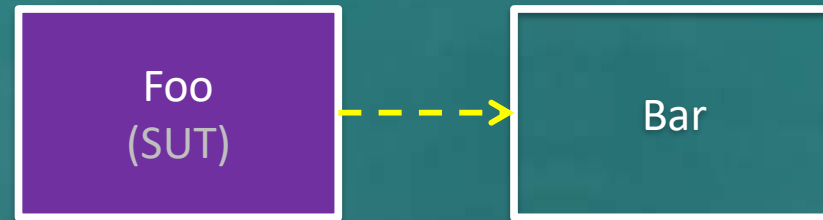
1. Submit weekly quiz [P](#)
2. Submit midterm peer evaluations on TEAMMATES [🕒 Sun, Mar 16th 2359](#) [P](#)

iP:

1. Evaluate two peer iPs [🕒 Sat, Mar 15th 2359](#) [P](#)

Unit testing
Integration testing
System testing
Acceptance testing
Alpha/beta testing

Exploratory vs scripted testing



How can I test Foo without being affected by Bar?

→ Dependency Injection

Can we make Foo easier to test?

→ Testability

To what extent Foo is being tested?

→ Test coverage

Week 8

Topics:

- [W8.1] [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [W8.2] [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [W8.3] [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [W8.4] Testing: Types ⚡⚡
- [W8.5] Testing: Intermediate Concepts ⚡⚡

Dependency injection
Testability
Test coverage

Admin:

1. Submit weekly quiz [P](#)
2. Submit midterm peer evaluations on TEAMMATES [🕒 Sun, Mar 16th 2359](#) [P](#)

iP:

1. Evaluate two peer iPs [🕒 Sat, Mar 15th 2359](#) [P](#)

tP: [📦 v1.2](#)

1. 👤 Divide MVP features among members
2. 👤/👤 [Optional] Rename packages/classes
3. 👤 Add the first functionality increment

[🕒 Thu, Mar 13th 23:59](#)

Week 8

Topics:

- [W8.1] [Revisiting] Drawing Class/Object Diagrams - Basics ⚡
- [W8.2] [Revisiting] Drawing Class/Object Diagrams - Intermediate ⚡⚡
- [W8.3] [Revisiting] Drawing Sequence Diagrams ⚡⚡
- [W8.4] Testing: Types ⚡⚡
- [W8.5] Testing: Intermediate Concepts ⚡⚡

Admin:

1. Submit weekly quiz [P](#)
2. Submit midterm peer evaluations on TEAMMATES [🕒 Sun, Mar 16th 2359](#) [P](#)

iP:

1. Evaluate two peer iPs [🕒 Sat, Mar 15th 2359](#) [P](#)

tP: [📌 v1.2](#)

1. 👤 Divide MVP features among members
2. 👤/👤 [Optional] Rename packages/classes
3. 👤 Add the first functionality increment

[🕒 Thu, Mar 13th 23:59](#)



tP work division:

More learning,
less dependent on others

[] Recommended to divide by *feature*

[] Recommended to divide *components* (UI, Logic, Storage, etc.) among members

More expertise within
the team

debugging features ✓

code navigation features ✓

A sample of IntelliJ **code editing shortcuts**

What if not using IntelliJ?

Observe, and look for similar features in your code editor.

duke3 - Duke.java [duke3.main]

duke3 > src > main > java > duke > Duke > main

Project: README.md x Duke.java x

```
1 package duke;
2
3 public class Duke {
4     String s;
5     public static void main(String[] args) {
6         System.out.println("Hello friend!");
7         String logo = " _____ \n"
8             + "| _ _ \\ _ _ _ | | _ _ _ _ \n"
9             + "| | | | | | | | / / _ \\ \\ \n"
10            + "| | | | | | | | < _ _ / \n"
11            + "| _ _ / _ _ _ _ | | \\ \\ _ _ _ | \n";
12         System.out.println("Hello from\n" + logo + (new Duke()).s);
13         System.out.println("Hello again");
14     }
15 }
16
```

Insert a new line below current line

Normal way:

1. Go to end of current line
2. Press enter

Git: [status icons]

Event Log

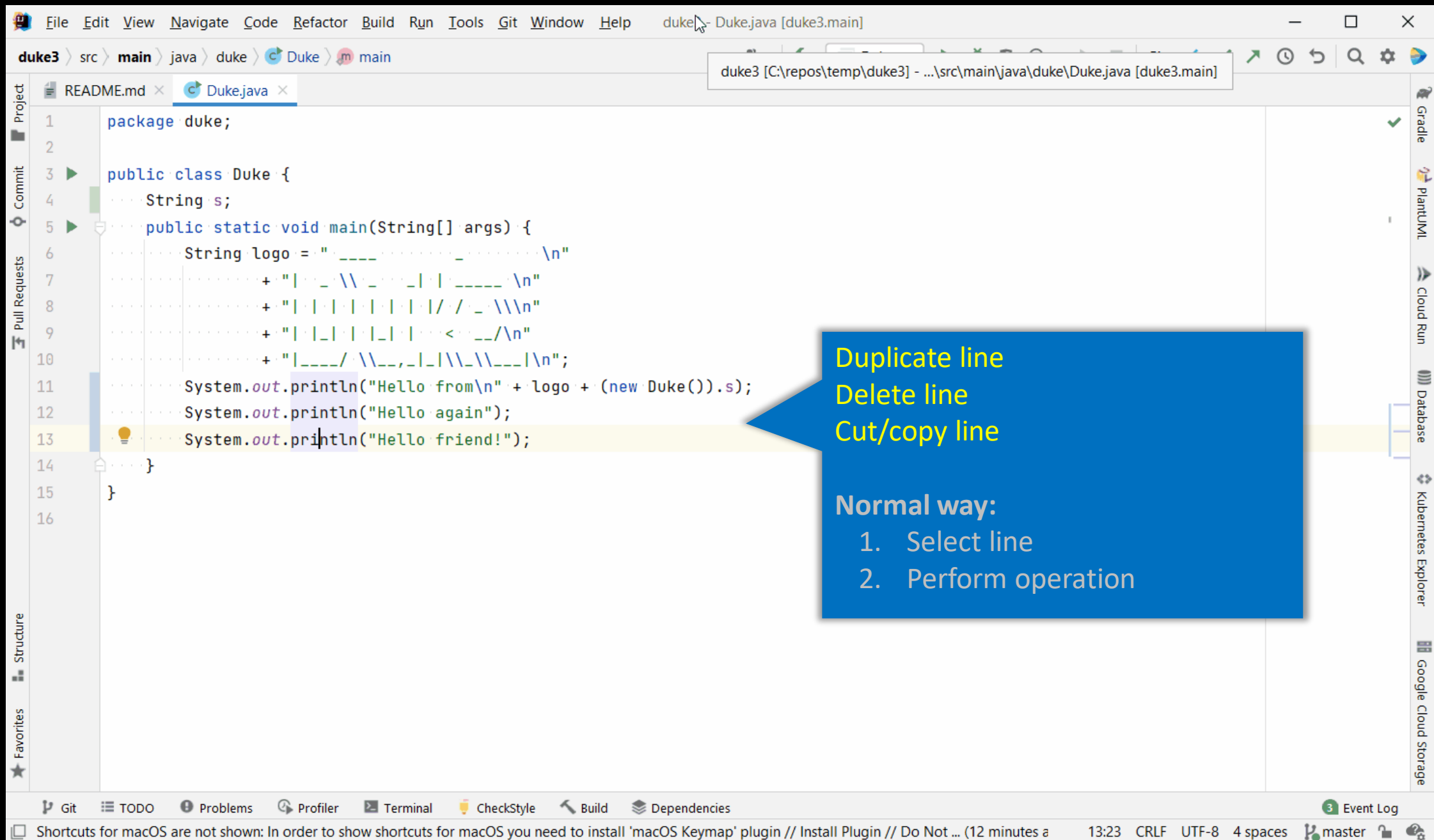
Shortcuts for macOS are not shown: In order to show shortcuts for macOS you need to install 'macOS Keymap' plugin // Install Plugin // Do Not Sho... (a minute : 6:23 CRLF UTF-8 4 spaces master [status icons])

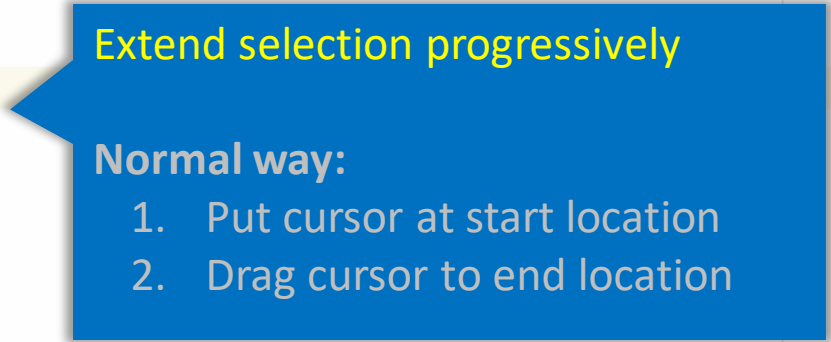




Normal way:

1. Select block
2. Cut
3. Go to new location
4. Create blank space
5. Paste





duke3 - Duke.java [duke3.main]

duke3 > src > main > java > duke > Duke > main

Project: README.md x Duke.java x

```
1 package duke;
2
3 public class Duke {
4     String s;
5     public static void main(String[] args) {
6         System.out.println("Hello again");
7         String logo = " _____ \n"
8             + "| _ _ \\ _ _ _ | | _ _ _ \n"
9             + "| | | | | | | | / / _ \\ \\ \n"
10            + "| | | | | | | | < _ _ / \n"
11            + "| _ _ / _ _ _ _ | | \\ \\ _ _ _ | \n";
12         System.out.println("Hello from\n" + logo + (new Duke()).s);
13         System.out.println("Hello Hello my friend!");
14     }
15 }
16
```

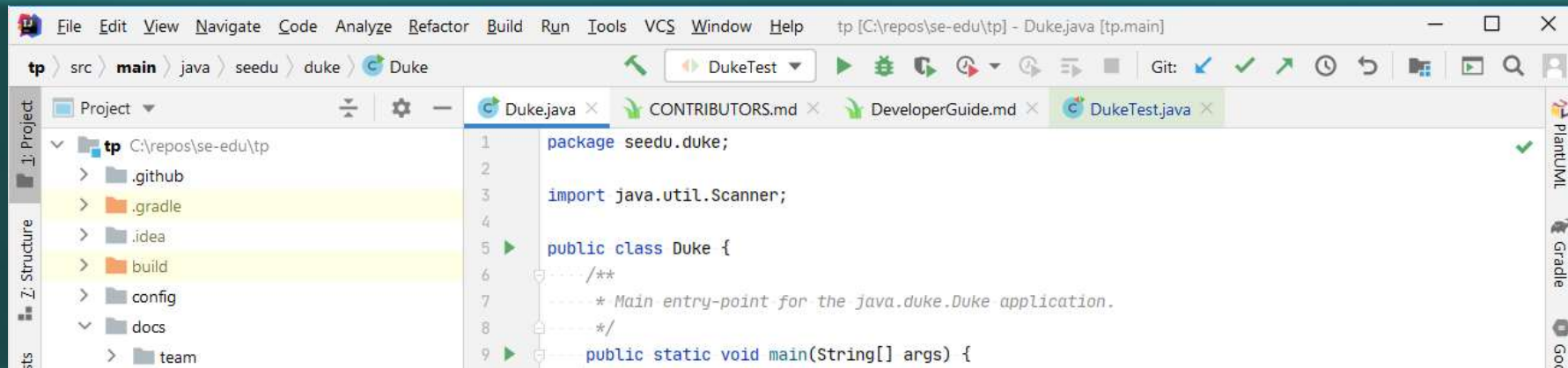
Gradle
PlantUML
Cloud Run
Database
Kubernetes Explorer
Google Cloud Storage

Structure
Favorites

Git
TODO
Problems
Profiler
Terminal
CheckStyle
Build
Dependencies
Event Log

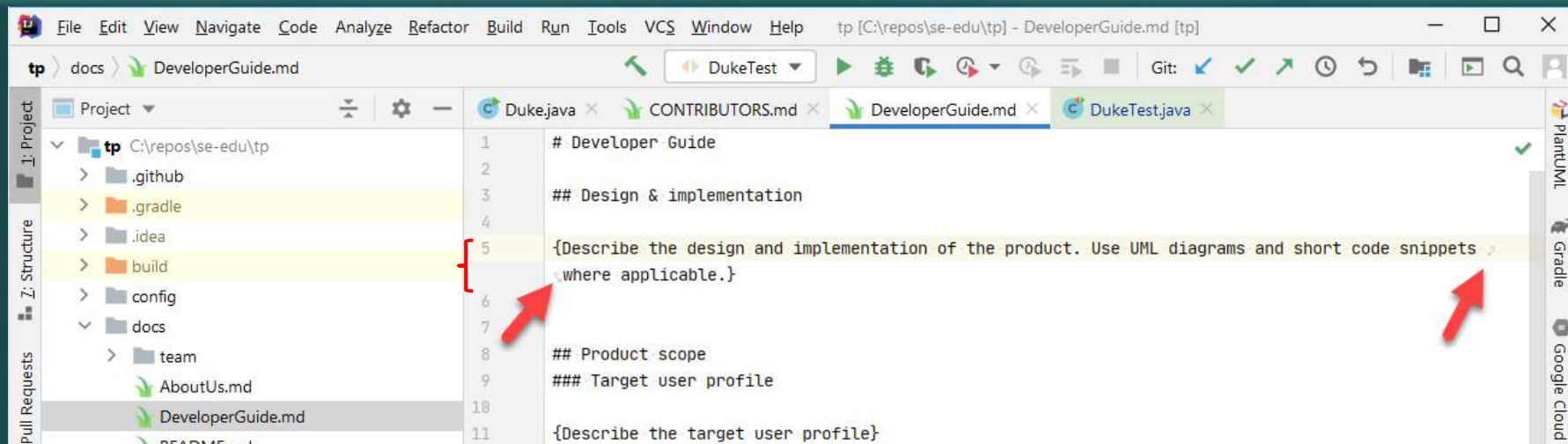
Shortcuts for macOS are not shown: In order to show shortcuts for macOS you need to install 'macOS Keymap' plugin // Install Plugin // ... (22 minutes a 6:34 (5 chars) CRLF UTF-8 4 spaces master

Select same text incrementally
Multi-cursor editing
Normal way: Find and replace



Insert line below/above, move line, delete line, incremental select, select similar, multi-cursor edits → **Just a sample only**; there are many more ...

To avoid horizontal scrolling



enable **soft-wrapping** long lines

✓ Revisit last week's topics
using examples/questions

✓ Preview of
next week's topics



*Thank
you!*

Dr Ganesh Neelakanta Iyer

gni@nus.edu.sg

<http://ganeshniyer.github.io>

<https://www.linkedin.com/in/ganeshniyer/>